

Problem marszrutyzacji i algorytm genetyczny



15.06.2023

Seminarium

„Technologie przyszłości: transformacja
cyfrowa”

Grzegorz Madejski

Zakład Sztucznej Inteligencji, Instytut Informatyki UG

Kilka słów wstępu...

- Wiele firm transportowych/logistycznych szuka wygodnych narzędzi do zarządzania flotą pojazdów, optymalnego wyznaczania tras.
- Istnieją gotowe narzędzia przeznaczone do realizacji tych celów, ale często nie spełniają w całości wymagań firmy.
- Potrzeba specjalistów tworzących narzędzia wykorzystujące algorytmy sztucznej inteligencji.
- W przypadku wyznaczania optymalnych tras dla pojazdów, dobrym modelem jest biologicznie inspirowany algorytm genetyczny.
- Celem seminarium jest zaznajomienie słuchaczy z podstawą działania algorytmu genetycznego i zademonstrowanie, jak poradzi sobie wyznaczaniem tras dla pojazdów w wirtualnym środowisku.

Część I: Algorytm genetyczny

Część II: Vehicle Routing Problem (Problem marszrutyzacji)

Kilka słów wstępu

- **Algorytm genetyczny** (genetic algorithm, GA) to algorytm metaheurystyczny służący do rozwiązywania problemów optymalizacyjnych z wykorzystaniem strategii inspirowanych biologicznie: ewolucji, selekcji naturalnej, mutacji, krzyżowania się.
- Algorytmy genetyczne rozwijały się w latach 50, 60, 70. Szczególnie wielki wkład w rozwój algorytmów genetycznych miał **John H. Holland** z University of Michigan.
- Początkowo algorytmy genetyczne operowały na rozwiązaniach kodowanych jako **ciągi binarne**. Obecnie, z uwagi na bardzo skomplikowane problemy, rozwiązania kodowane są na różne sposoby.



Zasada działania

Algorytm genetyczny operuje na rozwiązaniach jakiegoś problemu optymalizacyjnego. Rozwiązaniami mogą być najlepiej skonstruowane plany lekcji, najkrótsze ścieżki w grafie, najbardziej optymalne podziały jakichś dóbr.

1. Początkowo, algorytm **generuje** (zwykle losowe) **rozwiązania**, które nie są dobre.
2. Następnie **ocenia je** wg jakiegoś mądrego kryterium.
3. Potem na bazie wybranych, najlepiej ocenionych rozwiązań, **tworzy nowe rozwiązania**. Jest spora szansa, że nowe rozwiązania będą lepsze niż poprzednie!
4. Kroki 2 i 3 są powtarzane wielokrotnie, aż spełniony będzie jakiś warunek końcowy.

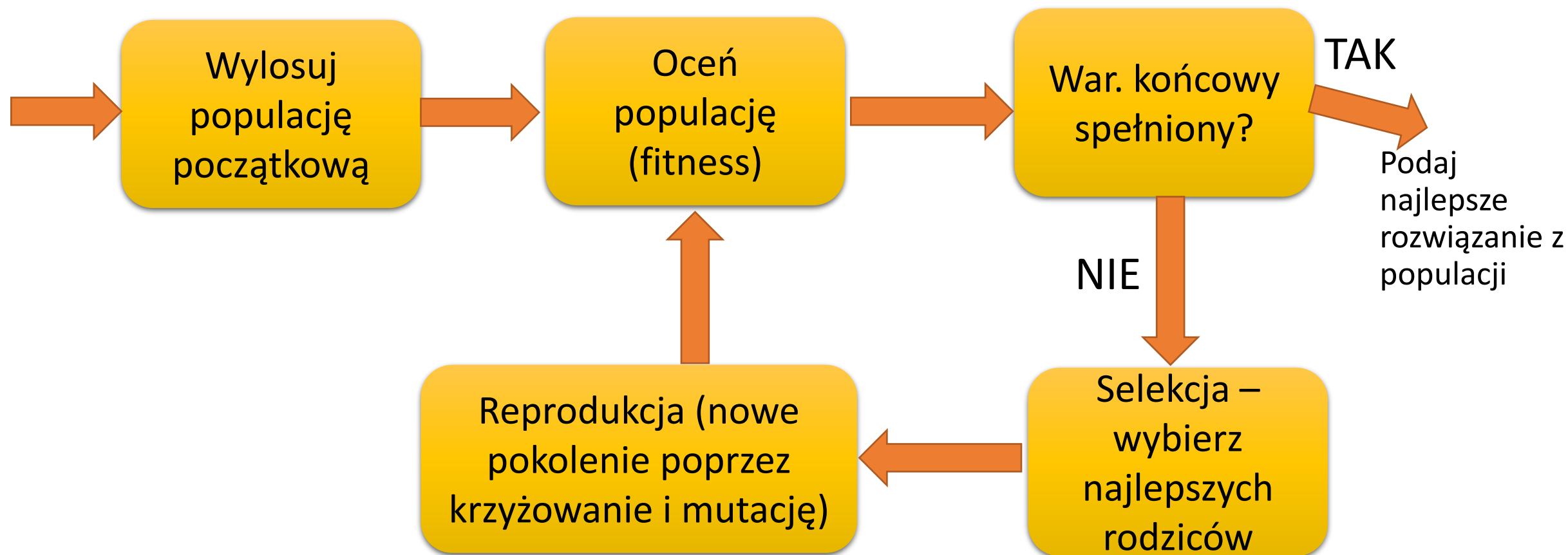
Zasada działania

W algorytmie tym stosowane są strategie inspirowane naturą. Przekładając algorytm na język biologiczny:

1. Początkowo, algorytm **generuje populację początkową**, w której znajduje się pewna ustalona liczba losowych **osobników**.
2. Następnie ocenia **stopień przystosowania** tych osobników do środowiska. Funkcję oceniającą często nazywa się **fitness**.
3. Osobniki są **selekcjonowane** i najsilniejsze przystępują do **reprodukcji**. Tworzą potomków poprzez **krzyżowanie** i **mutację**.
4. Powstaje **nowe pokolenie**. Idziemy do kroku 2, a potem 3, chyba że spełniony będzie jakiś warunek końcowy.

Zasada działania

Algorytm genetyczny na schemacie:



Problem plecakowy

Zilustrujmy działanie algorytmu genetycznego na przykładzie problemu plecakowego w wersji świątecznej:

*W supermarkecie w szczególnej promocji rozdawane są towary za darmo. Jedna osoba może wziąć tylko tyle towarów, ile zmieści się w koszyku o pojemności **250** dm³. Jakie towary wybrać, żeby było to najbardziej opłacalne?*

Nr	Przedmiot	Cena	Objętość
1	Termofor Lama	150	15
2	Odkurzacz Henry	750	125
3	Sofa na kółkach	5000	300
4	Maska Koń	40	16
5	Poduszka puchowa	100	21
6	Karabin na wodę	40	10
7	Zestaw słoików	60	12
8	Tablet	700	12
9	Plecak Pikachu	200	14
10	Pufa Grzyb	600	80

Nr	Przedmiot	Cena	Objętość
11	Robot koszący	1000	100
12	Globus	215	70
13	Kuweta	150	60
14	Cienie do makijażu	100	6
15	Piłka nożna	60	27
16	Piórnik Hotdog	10	2
17	Książka Algorytmy	40	2
18	Patelnia Żaba	40	8
19	Głośniki	350	40
20	Komplet sztucców	600	18

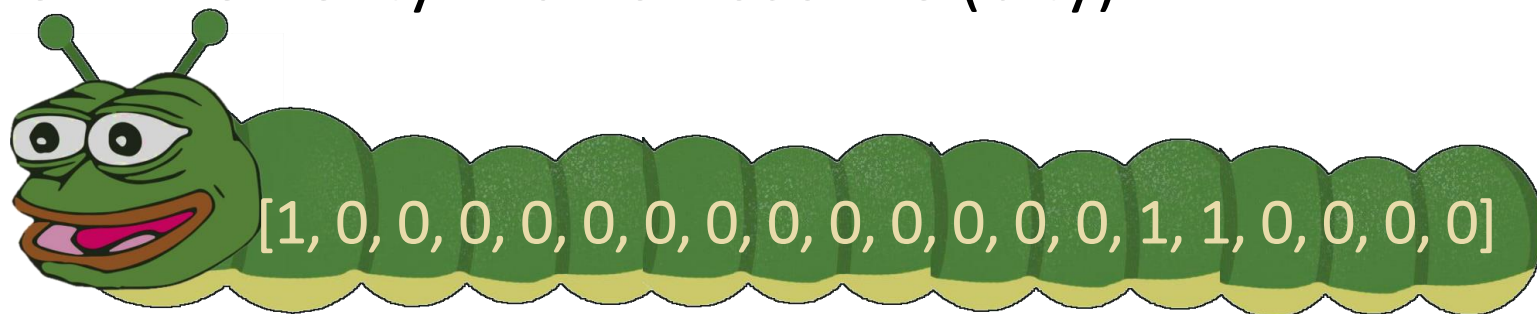


Rozwiązania, osobniki, chromosomy

- Pierwszym poważnym wyzwaniem w rozwiązywaniu problemu jest ustalenie kodowania rozwiązań.
- Klasycznie zakodowane rozwiązanie powinno mieć postać ciągu (wektora) binarnego o ustalonej długości. Przykład takiego ciągu:
[0, 1, 0, 0, 1, 0, 1, 0, 0]
- Czy da się to zrobić dla naszego problemu plecakowego?

Rozwiązania, osobniki, chromosomy

- TAK! Mamy 20 przedmiotów. Tworzymy ciągi binarne o długości 20, w których **zera** wskazują, że przedmiot o danym indeksie **nie jest brany** do koszyka. A **jedynki** wskazują, że dany przedmiot jest **zabierany**.
- Jeśli osoba weźmie przedmioty [termofor, piłka, piórnik], o indeksach [1, 15, 16]. To rozwiązanie można zakodować jako:
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0]
- O całym rozwiązaniu mówimy, że jest **osobnikiem (individual)**, o wektorze binarnym mówimy, że jest **chromosomem** i często utożsamiamy go z osobnikiem. Elementy w chromosomie (bity) nazywamy zaś **genami**.



Fitness, czyli ocena przystosowania

- Kolejnym ważnym i trudnym krokiem w projektowaniu algorytmu genetycznego jest napisanie funkcji **fitness**.
- Funkcja ta musi sensownie oceniać każdego osobnika, patrząc na jego chromosom, który dostaje jako argument wejściowy.
- Oceny mogą być wystawiane w dowolnej skali liczbowej (o ile nie jest mała i dyskretna, wtedy słabo zadziała).
- Funkcja musi być zadana takim wzorem, aby naprowadzała algorytm genetyczny na najlepsze rozwiązanie. Jeśli jej wartość osiągnie maksimum, to znaczy że jej argument (osobnik) jest najlepszym rozwiązaniem.
- Jaką funkcję wybrać dla problemu plecakowego?

Fitness, czyli ocena przystosowania

- Jest wiele dobrych funkcji fitness do oceniania rozwiązań, ale najprostszą sensowną jest następująca:

```
fitness(chromosom)
    sum_cen = 0
    suma_obj = 0
    for i=1 to 20 do
        if chromosom[i] == 1 then
            sum_cen = sum_cen + cena[i]
            suma_wag = suma_wag + waga[i]
    if suma_wag <= 250 then
        return sum_cen
    else
        return 0
```

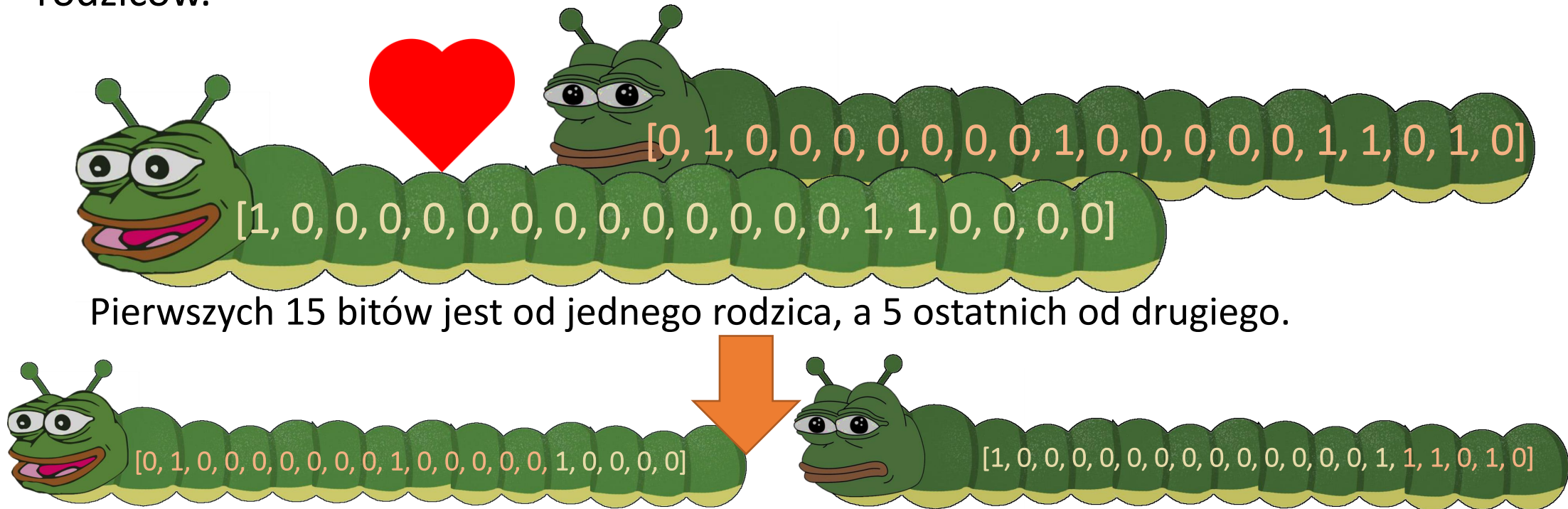
Selekcja rodziców

Po ocenieniu osobników należy przeprowadzić selekcję na to, kto zostanie rodzicem. Jest na to kilka technik m.in.:

- **Selekcja stabilnego stanu** (steady state selection, sss) - wybierz pewien procent najlepszych rodziców, wyprodukuj z nich potomków dobierając losowo w pary. Wymień najgorsze chromosomy na powstałe dzieci.
- **Metoda turniejowa** - wybierz losowo grupę kilku chromosomów, z tej grupy najlepszy jest selekcjonowany na rodzica.
- **Metoda ruletki** - chromosomy grają w ruletkę, ale im lepsze mają fitness, tym większe ich pole wygranej w ruletce (większa szansa na wygraną i wylosowanie do reprodukcji).

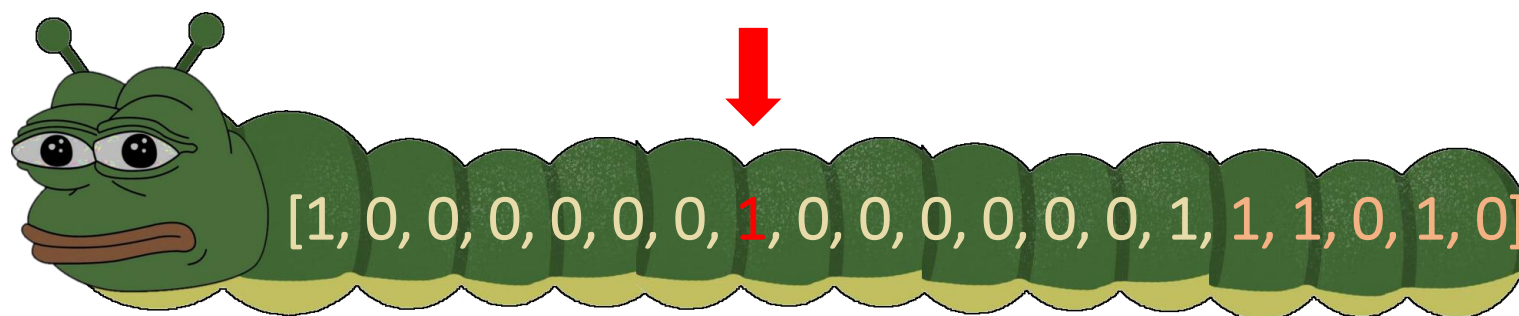
Reprodukcja

- Po wyselekcjonowaniu rodziców, łączeni są oni w pary i tworzą potomków w procesie **krzyżowania** (crossover).
- W przypadku chromosomów o prostej strukturze, robi się zazwyczaj **krzyżowanie jednopunktowe** (single-point crossover) lub wielopunktowe. Polega ono na rozkrojeniu rodziców w losowym punkcie. Dwójka dzieci powstaje z kawałków rodziców.



Reprodukcja

- Powstałe dzieci są poddawane operatorowi mutacji. Niektóre geny mogą zmienić wartość.
- W klasycznym chromosomie binarnym losowane są bity (niewielki procent). Wylosowane bity zmieniają wartość na przeciwną.
- Mutacja jest pożyteczna – pozwala zachować genetyczną różnorodność w populacji i uciekać z maksimów lokalnych funkcji fitness.



Nowe pokolenie

- Reprodukcyjne powtarzamy, aż otrzymamy wystarczającą liczbę potomków do uzupełniania pokolenia.
- Część rodziców (tzw. elita) może przeżyć z dziećmi do nowego pokolenia.
- Algorytm ponownie ocenia populację... cykl się powtarza.
- Mutacja jest pożyteczna – pozwala zachować genetyczną różnorodność w populacji i uciekać z maksimów lokalnych funkcji fitness.

Lokalne maksima (minima)

- Częstym problemem algorytmów genetycznych jest zbieganie do lokalnego maksimum (minimum) funkcji fitness i otrzymanie suboptymalnego rozwiązania.
- Szansa na znalezienie globalnego maksimum dla większych problemów jest mała. Istnieje jednak kilka technik, które zapobiegają utykaniu w minimach lokalnych.
- Pomocne może być **zwiększenie mutacji** (więcej losowych genów).
- Usuwanie lub karanie zduplikowanych chromosomów – będzie miejsce na inne (**niche penalty**).
- Dodawanie co jakiś czas losowych chromosomów (**random immigrants**).
- Zastosowanie **modelu wyspowego**, w którym populacja dzielona jest na podpopulacje zamieszkujące różne wyspy. Populacje co jakiś czas wymieniają się osobnikami.
- Wykorzystanie algorytmów hybrydowych (genetyczny + dodatkowy algorytm) lub innych metaheurystycznych (np. Tabu Search).

gen-xmas.py

- Uruchomimy program w Pythonie, wykorzystujący paczkę pygad.
- Program rozwiąże problem plecakowy. Zobaczymy czy skutecznie...

```
1  import pygad
2  import numpy
3
4  #define problem
5  przedm = ["Termofor Lama", "Odkurzacz Henry", "Sofa na kołkach", "Maska Kon", "Poduszka puchowa", "Karabin na wo
6  "Plecak Pikachu", "Pufa Grzyb", "Robot koszacy", "Globus", "Kuweta", "Cienie do makijazu", "Pilka nozna", "Piorn
7  "Patelnia Zaba", "Glosniki", "Komplet sztuccow"]
8  ceny = [150, 750, 5000, 40, 100, 40, 60, 700, 200, 600, 1000, 215, 150, 100, 60, 10, 40, 40, 350, 600]
9  obj = [15, 125, 300, 16, 21, 10, 12, 12, 14, 80, 100, 70, 60, 6, 27, 2, 2, 8, 40, 18]
10 limit = 250
11
12 # define all possible genes
13 gene_space = [0, 1]
14
15 #define the number of genes in a chromosome/solution
16 num_genes = 20
17
```

gen-xmas.py

```
17
18 # define the fitness function
19 # how to grade chromosomes?
20 def fitness_func(solution, solution_idx):
21     suma_cen = numpy.sum(solution * ceny)
22     suma_obj = numpy.sum(solution * obj)
23     if suma_obj > limit:
24         return 0
25     else:
26         return suma_cen
27
28 fitness_function = fitness_func
29
30 # size of the population
31 sol_per_pop = 30
32
33 # part of the population to become parents
34 num_parents_mating = 15
35
36 # how many parents will live with children
37 keep_parents = 3
38
```

gen-xmas.py

```
39     # maximal number of generations
40     num_generations = 50
41
42
43     # type of selection
44     parent_selection_type = "sss"
45
46     # type of crossover
47     crossover_type = "single_point"
48
49     # how should mutation work?
50     # set the percentage to: round_up(100/num_genes)
51     mutation_type = "random"
52     mutation_percent_genes = 5
```

gen-xmas.py

```
53  
54 # initiate the algorithm, gice all the parameters from above  
55 ga_instance = pygad.GA(gene_space=gene_space,  
56                         num_generations=num_generations,  
57                         num_parents_mating=num_parents_mating,  
58                         fitness_func=fitness_function,  
59                         sol_per_pop=sol_per_pop,  
60                         num_genes=num_genes,  
61                         parent_selection_type=parent_selection_type,  
62                         keep_parents=keep_parents,  
63                         crossover_type=crossover_type,  
64                         mutation_type=mutation_type,  
65                         mutation_percent_genes=mutation_percent_genes)  
66  
67 # run the algorithm, it will start the generation cycle  
68 ga_instance.run()
```

gen-xmas.py

```
69
70 # summary: we return the solution
71 solution, solution_fitness, solution_idx = ga_instance.best_solution()
72 print("Parameters of the best solution : {solution}".format(solution=solution))
73 print("Fitness value of the best solution = {solution_fitness}".format(solution_fitness=solution_fitness))
74
75 print("Items to take:")
76 for i in range(20):
77     if solution[i] == 1:
78         print(przedm[i])
79
80 # show a plot of the best solution fitness throughout the generations
81 ga_instance.plot_fitness()
```

gen-xmas.py

Parameters of the best solution : [1. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 0. 0. 1. 0. 1. 1. 0. 0. 1.]

Fitness value of the best solution = 3400.0

Items to take:

Termofor Lama

Tablet

Plecak Pikachu

Pufa Grzyb

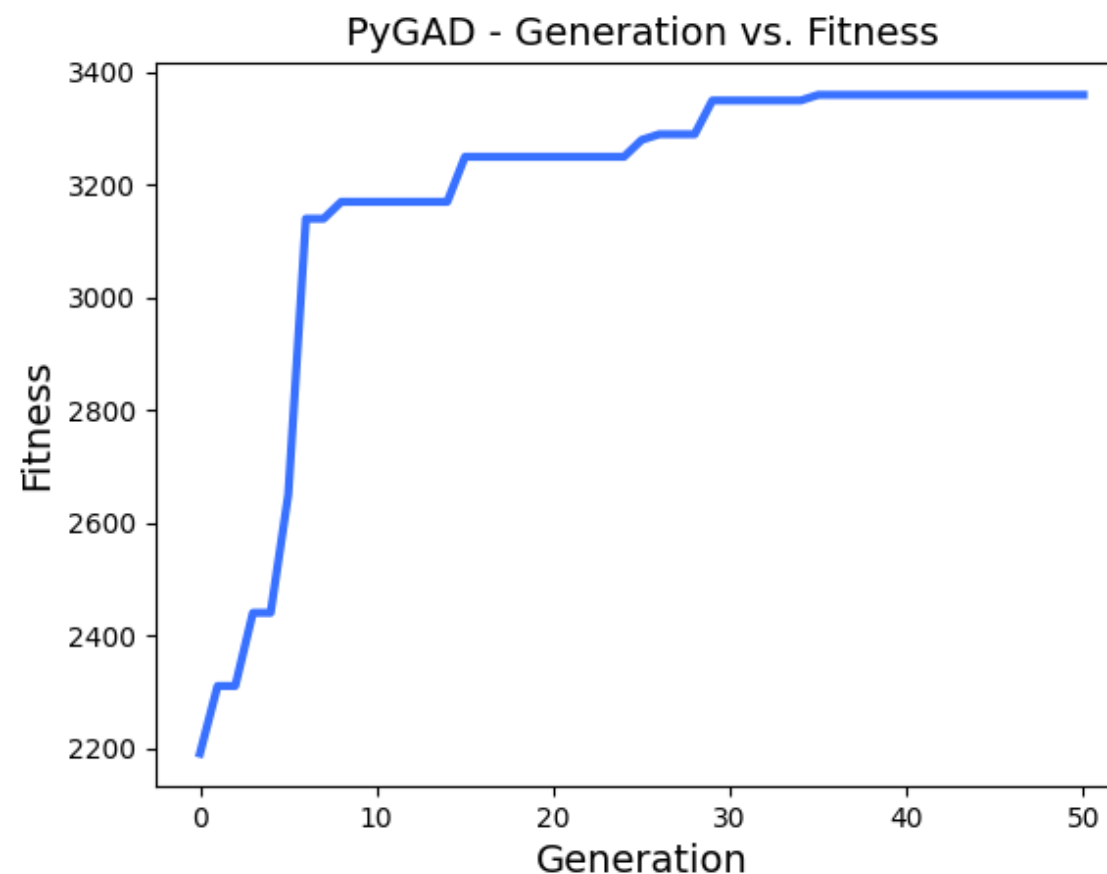
Robot koszacy

Cienie do makijazu

Piornik Hotdog

Ksiazka Algorytmy

Komplet sztuccow

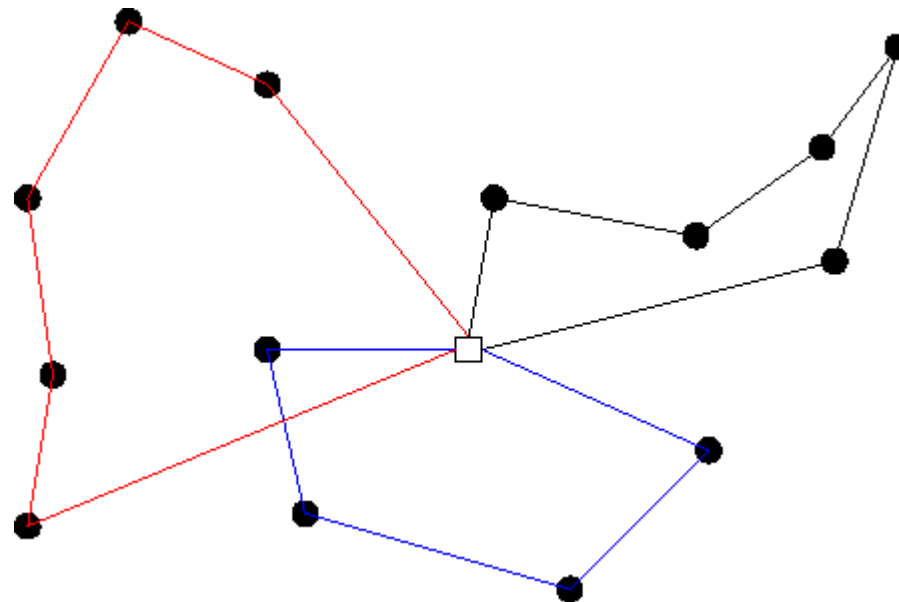


Część I: Algorytm genetyczny

Część II: Vehicle Routing Problem (Problem marszrutyzacji)

Definicja VRP

Vehicle Routing Problem (VRP), po polsku zwany **problemem marszrutyzacji**, to problem optymalizacyjny, w którym mamy bazę (depot) z pewną liczbą pojazdów oraz mamy klientów, którym trzeba dostarczyć towary z naszej bazy, a następnie do niej wrócić. Jakimi trasami powinny jechać pojazdy, by rozdały wszystkim klientom towary w najbardziej optymalny sposób?



Definicja VRP

Co rozumiemy przez pojęciem „**najbardziej optymalny sposób**”? Można to interpretować według swoich potrzeb. Najczęstsze interpretacje:

- Pojazdy przejechały łącznie najkrótszą drogę
- Pojazdy wygenerowały jak najmniejsze koszty (paliwo, opłaty, zakwaterowanie, kary za opóźnienia, itp.)
- Pojazdy rozwiozły towary w najkrótszym czasie. Lub inaczej mówiąc: podział pracy na pojazdy był równy i sprawiedliwy - każdy wyjeździł tyle samo godzin.
- Jeśli nie jesteśmy w stanie dotrzeć do wszystkich klientów (np. ze względu na ograniczenia pojazdów), to jakie trasy zmaksymalizują profity? Których klientów wybierać, a o których zapomnieć?

Definicja VRP

VRP jest właściwie całą klasą problemów. Można nakładać na niego przeróżne ograniczenia tworząc nowe warianty, np.

- CVRP (**Capacitated Vehicle Routing Problem**) – pojazdy mają ograniczoną ładowność.
- MDVRP (**Multi-Depot Vehicle Routing Problem**) – istnieje wiele baz pojazdów, mogą zacząć w jednej i skończyć w drugiej
- VRPTW (**Vehicle Routing Problem with Time Windows**) – klientów trzeba odwiedzić w określonych okienkach czasowych.
- VRPP (**Vehicle Routing Problem with Profits**) – zakładając, że nie wszystkich odwiedzimy, to maksymalizować profity
- VRPPD (**Vehicle Routing Problem with Pickup and Delivery**) – każdy towar ma lokację odbioru i lokację docelową

VRP i algorytm genetyczny

- Do dzisiejszych demonstracji wybierzemy MDVRP (**Multi-Depot Vehicle Routing Problem**), czyli dopuszczamy wiele baz dla samochodów , ale z zastrzeżeniem, że baza startowa i końcowa dla każdego z nich musi być taka sama.
- Nie są dopuszczalne tzw. **inter-depot routes**.
- Intuicja za tym jest taka, że pojazdy mogą należeć do różnych podwykonawców (firm spedycyjnych lub transportowych), albo nawet być prywatnymi samochodami, które wracają do domu.
- W przyszłości, pewnie wezmę bardziej praktyczne i bardziej ograniczone wersje problemu np. CMDVRPTWPPD (**Capacitated Multi-Depot Vehicle Routing Problem with Time Windows, Profits, and Pickup and Delivery**).

VRP i algorytm genetyczny

Dokładniejsza specyfikacja naszego problemu:

- **Zbiór klientów**, zakodowanych jako punkty współrzędnych na mapie (x, y). Będą to współrzędne geograficzne mniej więcej w granicach Polski. Każdy klient dostaje też swoje ID – liczbę naturalną.
- **Zbiór baz (depotów)** – zakodowanych analogicznie jak wyżej. Dodatkowo każda baza ma przyporządkowaną pewną liczbę samochodów.
- Wynikiem działania będzie **sekwencja klientów dla każdego pojazdu**.
- Moim kryterium optymalności będzie przede wszystkim **minimalizowanie sumarycznej drogi wszystkich samochodów** (co przełoży się też na minimalizowanie kosztów za paliwo).
- Zaproponuję też pewną modyfikację, która może też lekko premiować **jak najszybsze rozwiązanie towarów** (czas).
- Nie wprowadzam ograniczeń na odległości przejazdu pojazdów/czas pracy.

VRP i algorytm genetyczny

Jednym z najważniejszych kroków jest ustalenie struktury chromosomu.

- Jeśli mamy dwa pojazdy, odwiedzające 5 klientów, to rozwiązaniem są dwie listy np. [0, 2, 4], [1, 3].
- Jednak jak krzyżować takie rozwiązanie? Np. z innym: [0, 3], [1, 2, 4].
- Jest to problematyczne. Wymaga skonstruowania własnej funkcji do krzyżowania, która działa w wyspecjalizowany sposób i sprawdza, czy nowe trasy są legalne, i je naprawia (tabelki z <https://iopscience.iop.org/article/10.1088/1757-899X/325/1/012012/pdf>) – Partially Mapped Crossover.

Table 2. Parent 1 & Parent 2

Vehicle. 1					Vehicle. 2					Vehicle. 3					Vehicle. 4					Vehicle. 5				
B	5	18	2	17	B	10	20	1	9	A	12	14	16	6	A	4	8	7	19	B	15	13	11	3
B	14	13	6	19	B	10	8	16	9	B	3	11	7	20	B	17	1	5	18	B	15	4	2	12

Table 3. Descendant 1 & Descendant 2 :

Vehicle. 1					Vehicle. 2					Vehicle. 3					Vehicle. 4					Vehicle. 5				
B	5	2	6	17	A	10	20	1	9	B	12	14	16	4	B	8	7	19	18	B	15	13	11	3
B	14	13	2	6	A	10	8	16	9	B	3	11	7	20	B	17	1	5	19	A	18	15	4	12

VRP i algorytm genetyczny

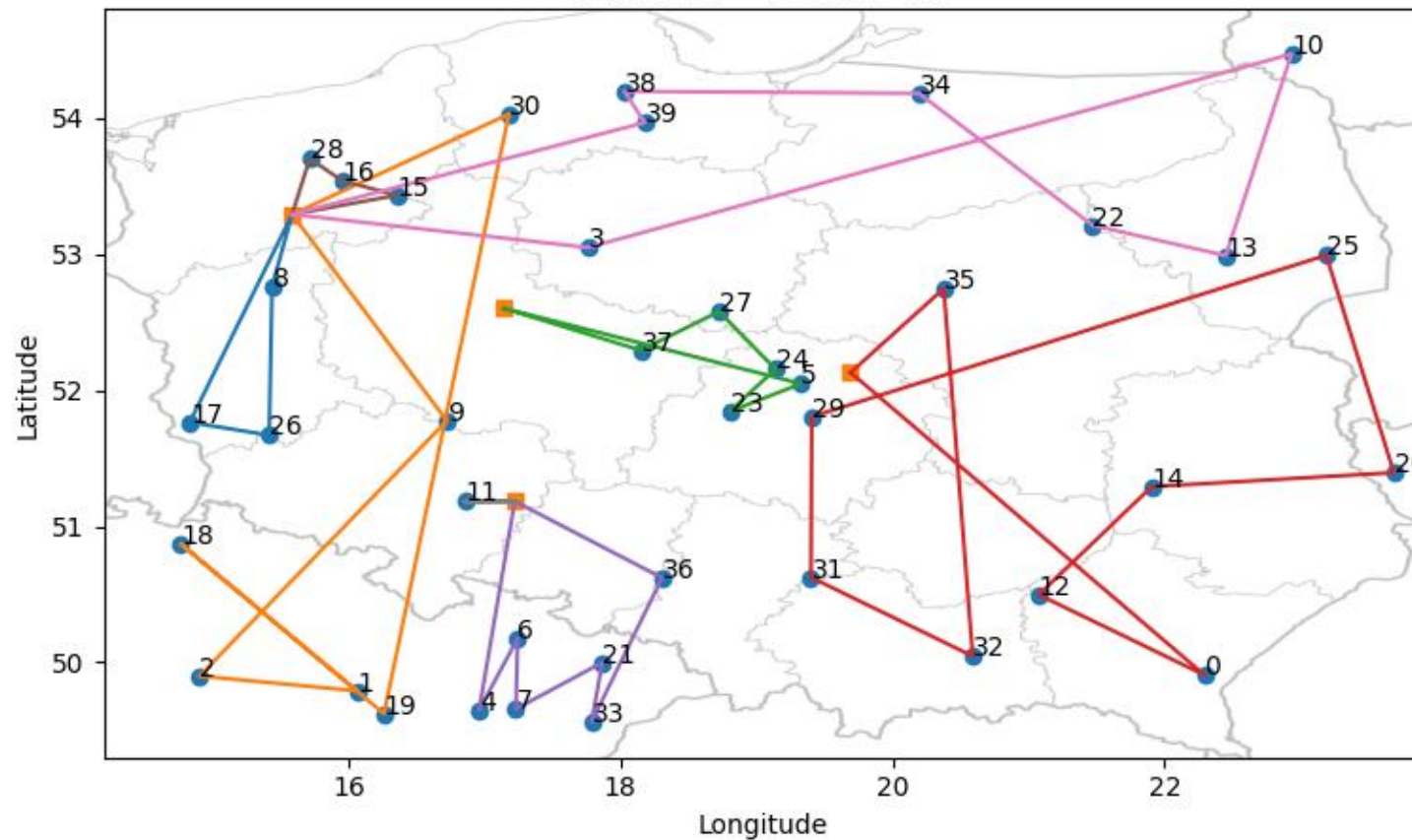
Przyjąłem prostsze podejście.

- Chromosom, to zestaw liczb rzeczywistych, o długości takiej jak liczba klientów. Każda liczba w chromosomie wskazuje, jaka ciężarówka i w jakiej kolejności odwiedzi klienta o danym indeksie.
- Część całkowita liczby wskazuje nam id pojazdu, który obsługuje danego klienta.
- Część ułamkowa liczby wskazuje w jakiej kolejności dana ciężarówka odwiedzi miasta (wg kolejności, od największego do najmniejszego)
- Przykład $[1.3, 0.5, 0.9, 1.7, 1.2]$ oznacza, że pojazd 0 odwiedzi klientów $[2, 1]$, a pojazd 1 klientów $[3, 0, 4]$.
- Dzięki takiemu podejściu krzyżowania i mutacja są proste (podobne jak dla chromosomów binarnych).
- Funkcja fitness zaś minimalizuje sumę wszystkich przejechanych tras.

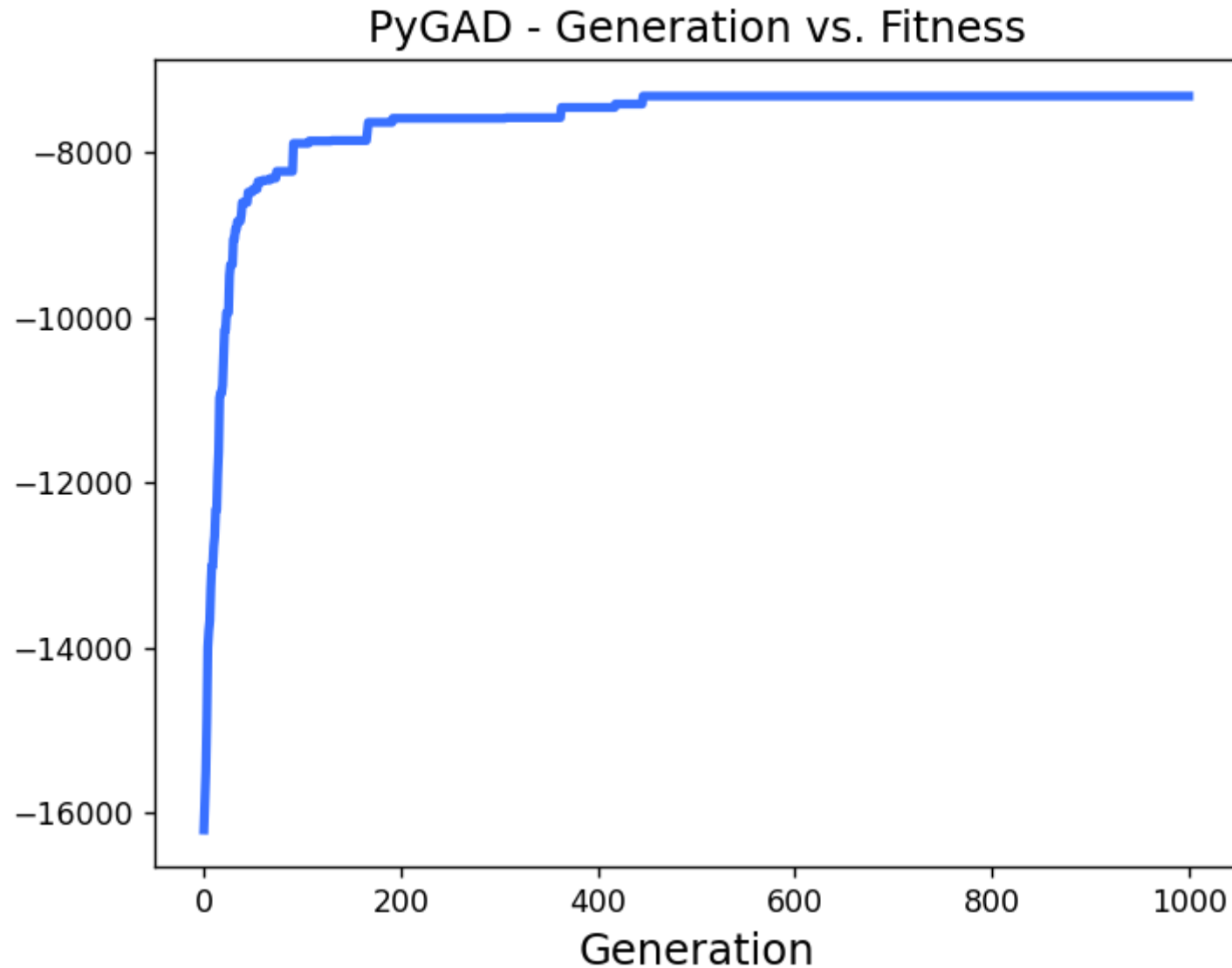
VRP i algorytm genetyczny

Genetic algorithm

Path cost = 7800.16 km



VRP i algorytm genetyczny



Godna polecenia książka:

„Genetic Algorithms in Search, Optimization & Machine Learning” David E. Goldberg.